

=== GNB (1785511165) ===

This configuration file example shows how to configure the srsRAN Project gNB to allow srsUE to connect to it.
This specific example uses ZMQ in place of a USRP for the RF-frontend, and creates an FDD cell with 10 MHz bandwidth
To run the srsRAN Project gNB with this config, use the following command:

sudo ./gnb -c gnb_zmq.yaml

```
cu_cp:
  amf:
    addr: 10.53.1.2          # The address or hostname of the AMF.
    port: 38412
    bind_addr: 10.53.1.1     # A local IP that the gNB binds to for traffic from the AMF.
    supported_tracking_areas:
      - tac: 7
      plmn_list:
        - plmn: "21401"
          tai_slice_support_list:
            - sst: 1
    inactivity_timer: 7200    # Sets the UE/PDU Session/DRB inactivity timer to 7200 seconds. Supported: [1 - 7200]

ru_sdr:
  device_driver: zmq         # The RF driver name.
  device_args: tx_port=tcp://127.0.0.1:2000,rx_port=tcp://127.0.0.1:2001,base_srate=11.52e6 # Optionally pass argument
  srate: 11.52              # RF sample rate might need to be adjusted according to selected bandwidth.
  tx_gain: 75               # Transmit gain of the RF might need to adjusted to the given situation.
  rx_gain: 75               # Receive gain of the RF might need to adjusted to the given situation.

cell_cfg:
  dl_arfcn: 630000          # ARFCN of the downlink carrier (center frequency).
  band: 78                  # The NR band.
  channel_bandwidth_MHz: 20 # Bandwidth in MHz. Number of PRBs will be automatically derived.
  common_scs: 15            # Subcarrier spacing in kHz used for data.
  plmn: "21401"             # PLMN broadcasted by the gNB.
  tac: 7                    # Tracking area code (needs to match the core configuration).
  pdccch:
    common:
      ss0_index: 0          # Set search space zero index to match srsUE capabilities
      coresets0_index: 6    # Set search CORESET Zero index to match srsUE capabilities
    dedicated:
      ss2_type: common      # Search Space type, has to be set to common
      dci_format_0_1_and_1_1: false # Set correct DCI format (fallback)
  prach:
    prach_config_index: 1    # Sets PRACH config to match what is expected by srsUE
  pdsch:
    mcs_table: qam64         # Sets PDSCH MCS to 64 QAM
  pusch:
    mcs_table: qam64         # Sets PUSCH MCS to 64 QAM

log:
  filename: /tmp/gnb.log     # Path of the log file.
  all_level: info            # Logging level applied to all layers.
  hex_max_size: 0

pcap:
  mac_enable: false         # Set to true to enable MAC-layer PCAPs.
  mac_filename: /tmp/gnb_mac.pcap # Path where the MAC PCAP is stored.
  ngap_enable: false        # Set to true to enable NGAP PCAPs.
  ngap_filename: /tmp/gnb_ngap.pcap # Path where the NGAP PCAP is stored.
  e2ap_enable: true         # Set to true to enable E2AP PCAPs.
  e2ap_du_filename: /tmp/gnb_du_e2ap.pcap # Path where the DU E2AP PCAP is stored.
  e2ap_cu_cp_filename: /tmp/gnb_cu_cp_e2ap.pcap # Path where the CU-CP E2AP PCAP is stored.
  e2ap_cu_up_filename: /tmp/gnb_cu_up_e2ap.pcap # Path where the CU-UP E2AP PCAP is stored.

#e2:
# enable_du_e2: true        # Enable DU E2 agent (one for each DU instance)
# e2sm_kpm_enabled: true    # Enable KPM service module
```

```
# e2sm_rc_enabled: true          # Enable RC service module
# addr: 127.0.0.1                # RIC IP address
#bind_addr: 127.0.0.100         # A local IP that the E2 agent binds to for traffic from the RIC. ONLY required if
# port: 36421                    # RIC port

metrics:
  layers:
    enable_rlc: true            # Enable RLC metrics
    enable_sched: true          # Enable DU scheduler metrics
  periodicity:
    du_report_period: 1000      # Set DU statistics report period to 1s
```

=== UE (1785511165) ===

```
[rf]
freq_offset = 0
tx_gain = 50
rx_gain = 40
srate = 11.52e6
nof_antennas = 1

device_name = zmq
device_args = tx_port=tcp://127.0.0.1:2001,rx_port=tcp://127.0.0.1:2000,base_srate=11.52e6

[rat.eutra]
dl_eurfcn = 2850
nof_carriers = 0

[rat.nr]
bands = 78
nof_carriers = 1

max_nof_prb = 106
nof_prb = 106

[pcap]
enable = none
mac_filename = /tmp/ue_mac.pcap
mac_nr_filename = /tmp/ue_mac_nr.pcap
nas_filename = /tmp/ue_nas.pcap

[log]
all_level = info
phy_lib_level = none
all_hex_limit = 32
filename = /tmp/ue.log
file_max_size = -1

[usim]
mode = soft
algo = milenage
opc = 63BFA50EE6523365FF14C1F45F88737D
k = 44556677889900aabbccddeeff112233
imsi = 214010000000005
imei = 353490069873319

[rrc]
release = 15
ue_category = 4

[nas]
apn = srsapn
apn_protocol = ipv4

[gw]
netns = ue1
ip_devname = tun_srsue
ip_netmask = 255.255.255.0

[gui]
enable = false
```

=== DOCKER (1785511165) ===

```
#
# Copyright 2021-2026 Software Radio Systems Limited
#
# This file is part of srsRAN
#
# srsRAN is free software: you can redistribute it and/or modify
# it under the terms of the GNU Affero General Public License as
# published by the Free Software Foundation, either version 3 of
# the License, or (at your option) any later version.
#
# srsRAN is distributed in the hope that it will be useful,
# but WITHOUT ANY WARRANTY; without even the implied warranty of
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
# GNU Affero General Public License for more details.
#
# A copy of the GNU Affero General Public License can be found in
# the LICENSE file in the top-level directory of this distribution
# and at http://www.gnu.org/licenses/.
#
```

services:

```
5gc:
  container_name: open5gs_5gc
  build:
    context: open5gs
    target: open5gs
    args:
      OS_VERSION: "22.04"
      OPEN5GS_VERSION: "v2.7.0"
  environment:
    - MONGODB_IP=127.0.0.1
    - OPEN5GS_IP=10.53.1.2
    - UE_IP_BASE=10.45.0
    - UPF_ADVERTISE_IP=10.53.1.2
    - DEBUG=false
    - SUBSCRIBER_DB=214010000000005,44556677889900aabbccddeeff112233,opc,63BFA50EE6523365FF14C1F45F88737D,8000,9,10
    - NETWORK_NAME_FULL=srsRAN
    - NETWORK_NAME_SHORT=srsRAN
    - TZ=Europe/Madrid
  privileged: true
  ports:
    - "9898:9999/tcp"
  # Uncomment port to use the 5gc from outside the docker network
  # - "38412:38412/sctp"
  # - "2152:2152/udp"
  command: 5gc -c open5gs-5gc.yml
  healthcheck:
    test: [ "CMD-SHELL", "nc -z 127.0.0.20 7777" ]
    interval: 3s
    timeout: 1s
    retries: 60
  networks:
    ran:
      ipv4_address: ${OPEN5GS_IP:-10.53.1.2}

gnb:
  container_name: srsran_gnb
  # Build info
  image: srsran/gnb
  build:
    context: ..
    dockerfile: docker/Dockerfile
    args:
      OS_VERSION: "24.04"
```

```

# privileged mode is required only for accessing usb devices
privileged: true
# Extra capabilities always required
cap_add:
  - SYS_NICE
  - CAP_SYS_PTRACE
volumes:
  # Access USB to use some SDRs
  - /dev/bus/usb/:/dev/bus/usb/
  # Sharing images between the host and the pod.
  # It's also possible to download the images inside the pod
  - /usr/share/uhd/images:/usr/share/uhd/images
  # Save logs and more into gnb-storage
  - gnb-storage:/tmp
# It creates a file/folder into /config_name inside the container
# Its content would be the value of the file used to create the config
configs:
  - gnb_config.yml
  - gnb_compose_config.yml
# Customize your desired network mode.
# current network configuration creates a private network with both containers attached
# An alternative would be `network: host`. That would expose your host network into the container. It's the easiest
networks:
  ran:
    ipv4_address: ${GNB_IP:-10.53.1.3}
  metrics:
    ipv4_address: 172.19.1.3
# Start GNB container after 5gc is up and running
depends_on:
  5gc:
    condition: service_healthy
# Command to run into the final container
command: gnb -c /gnb_config.yml -c /gnb_compose_config.yml

configs:
  gnb_config.yml:
    file: ../configs/gnb_zmq.yaml # Path to your desired config file
  gnb_compose_config.yml:
    content: |
      cu_cp:
        amf:
          addr: ${OPEN5GS_IP:-10.53.1.2}
          bind_addr: 0.0.0.0
      metrics:
        autostart_stdout_metrics: true
        enable_json: true
      remote_control:
        bind_addr: 0.0.0.0
        enabled: true

volumes:
  gnb-storage:

networks:
  ran:
    ipam:
      driver: default
      config:
        - subnet: 10.53.1.0/24
  metrics:
    ipam:
      driver: default
      config:
        - subnet: 172.19.1.0/24

```

=== NOTAS (1785511165) ===

- PLMN (21401) es consistente en gNB, UE (IMSI) y SUBSCRIBER_DB del Core (Regla 1).
- dl_arfcn (630000) está dentro del rango para Banda 78 (Regla 2).
- common_scs es 15, consistente con ssb_scs por defecto (Regla 3).
- Puertos ZMQ cruzados correctamente entre gNB y UE (Regla 4).
- bind_addr en gnb_zmq.yaml es 10.53.1.1, y en gnb_compose_config.yml es 0.0.0.0 (Regla 5).
- El bloque ru_sdr se ha copiado exactamente del CAG (Regla 6).
- IMSI, K y OPC son idénticos y están en las posiciones correctas en UE y SUBSCRIBER_DB (Regla 7).
- Se ha mantenido srates: 11.52e6 en ru_sdr (Regla 6) a pesar de solicitar 20 MHz de ancho de banda.